מעבדה בתכנון ספרתי מתקדם

הכנה 4

מגישים:

אלעד אסבג – 318530482

יוסי ברים- 315512848

צוות 7

.1

הלוח שלנו מספק 4 ספרות ב7 סיגמנטס בעוד שאנו צריכים 6 ספרות בשביל השעות, דקות והשניות.

.2

א)

יש לנו 2 ספרות כאשר ה7-סיגמנט מחובר לשניהם ועובד בשיטת הקטודה משותפת וצריך לבחור לפי האנודה איזה לדים יפעלו כאשר ניתן לה 1 לוגי. כדי לשלוט על ספרה אחת בנפרד נפעיל את פין

CATכאשר הוא יהיה 0 הספרה השמאלית תהיה מוצגת. וכן להפך ב1 לוגי.

ב)

כדי לחבר את הSSD לבורד שלנו, נצטרך לחבר לPMODE שלנו . כאשר יספיק לנו

PMODE 1 בעל 8 כניסות. מה שכן נצטרך להפוך את הכניסות כי כל אחד פועל

בצורה הפוכה.

ג)

יש כל מיני שיקולים. אם מדובר בשעון נעדיף להראות את השעות והדקות, ואתה השניות בחיצוני. מישום שהוא פחות חשוב. אבל אם מדברים על טיימר שצריך דיוק כמו במיקרוגל, נרצה להראות את הדקות והשניות.

בכל מקרה אצלינו יהיה יותר נוח שהדקות והשניות יהיו בבור כדי לבדוק את התקינות בזמן סביר. ומשום שהם באותו הלוגיקה של מחזור של 59.

.3

א)

```
library IEEE;
use IEEE.STD_LOGIC_1164.All;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity BINARY_TO_7SEGMENT is
port(
   B_IN            : in    STD_LOGIC_VECTOR(3 downto 0);
   seven_segment   : out   STD_LOGIC_VECTOR(6 downto 0);
);

end BINARY_TO_7SEGMENT;

architecture BINARY_TO_7SEGMENT_ARC of BINARY_TO_7SEGMENT is

begin

process(B_in)
begin

    case B_IN is
        when "0000"  => seven_segment <= "0000001"; --- 0
        when "0001"  => seven_segment <= "1001111"; --- 1
        when "0010"  => seven_segment <= "0010010"; --- 2
        when "0011"  => seven_segment <= "0000110"; --- 3
        when "0100"  => seven_segment <= "1001100"; --- 4
        when "0101"  => seven_segment <= "0100100"; --- 5
        when "0110"  => seven_segment <= "0100000"; --- 6
        when "0111"  => seven_segment <= "0001111"; --- 7
        when "1000"  => seven_segment <= "0000000"; --- 8
        when "1001"  => seven_segment <= "0000100"; --- 9
        when others  => seven_segment <= "XXXXXXX"; --- error
    end case;

end process;

end BINARY_TO_7SEGMENT_ARC;
```

ב)

```
library IEEE;
use IEEE.STD_LOGIC_1164.All;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity BINARY_TO_7SEGMENT_SSD is
port(
   B_IN            : in    STD_LOGIC_VECTOR(3 downto 0);
   seven_segment   : out   STD_LOGIC_VECTOR(6 downto 0);
);

end BINARY_TO_7SEGMENT_SSD;

architecture BINARY_TO_7SEGMENT_SSD_ARC of BINARY_TO_7SEGMENT_SSD is

begin

process(B_in)
begin

    case B_IN is
        when "0000"  => seven_segment <= "1111110"; --- 0
        when "0001"  => seven_segment <= "0110000"; --- 1
        when "0010"  => seven_segment <= "1101101"; --- 2
        when "0011"  => seven_segment <= "1111001"; --- 3
        when "0100"  => seven_segment <= "0110011"; --- 4
        when "0101"  => seven_segment <= "1011011"; --- 5
        when "0110"  => seven_segment <= "1011111"; --- 6
        when "0111"  => seven_segment <= "1110000"; --- 7
        when "1000"  => seven_segment <= "1111111"; --- 8
        when "1001"  => seven_segment <= "1111011"; --- 9
        when others  => seven_segment <= "XXXXXXX"; --- error
    end case;

end process;

end BINARY_TO_7SEGMENT_SSD_ARC;
```

(λ

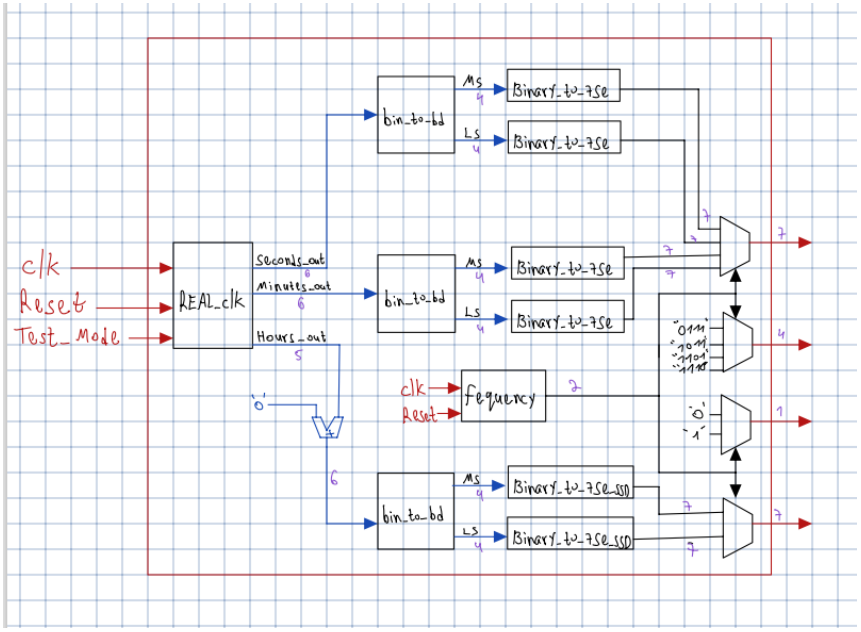
```
library IEEE;
use IEEE.STD_LOGIC_1164.All;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity BIN_TO_BD is
port(
   VALUE_BIN    : in    STD_LOGIC_VECTOR(5 downto 0);
   MS           : out   STD_LOGIC_VECTOR(3 downto 0);
   LS           : out   STD_LOGIC_VECTOR(3 downto 0)
);

end BIN_TO_BD;

architecture BIN_TO_BD_ARC of BIN_TO_BD is

begin

process(VALUE_BIN)
begin

    case VALUE_BIN is
        when "000000" => MS <= x"0" ;LS <= x"0" ;
        when "000001" => MS <= x"0" ;LS <= x"1" ;
        when "000010" => MS <= x"0" ;LS <= x"2" ;
        when "000011" => MS <= x"0" ;LS <= x"3" ;
        when "000100" => MS <= x"0" ;LS <= x"4" ;
        when "000101" => MS <= x"0" ;LS <= x"5" ;
        when "000110" => MS <= x"0" ;LS <= x"6" ;
        when "000111" => MS <= x"0" ;LS <= x"7" ;
        when "001000" => MS <= x"0" ;LS <= x"8" ;
        when "001001" => MS <= x"0" ;LS <= x"9" ;
        when "001010" => MS <= x"1" ;LS <= x"0" ;
        when "001011" => MS <= x"1" ;LS <= x"1" ;
        when "001100" => MS <= x"1" ;LS <= x"2" ;
        when "001101" => MS <= x"1" ;LS <= x"3" ;
        when "001110" => MS <= x"1" ;LS <= x"4" ;
        when "001111" => MS <= x"1" ;LS <= x"5" ;
        when "010000" => MS <= x"1" ;LS <= x"6" ;
        when "010001" => MS <= x"1" ;LS <= x"7" ;
        when "010010" => MS <= x"1" ;LS <= x"8" ;
        when "010011" => MS <= x"1" ;LS <= x"9" ;
        when "010100" => MS <= x"2" ;LS <= x"0" ;
        when "010101" => MS <= x"2" ;LS <= x"1" ;
        when "010110" => MS <= x"2" ;LS <= x"2" ;
        when "010111" => MS <= x"2" ;LS <= x"3" ;
        when "011000" => MS <= x"2" ;LS <= x"4" ;
        when "011001" => MS <= x"2" ;LS <= x"5" ;
        when "011010" => MS <= x"2" ;LS <= x"6" ;
        when "011011" => MS <= x"2" ;LS <= x"7" ;
        when "011100" => MS <= x"2" ;LS <= x"8" ;
        when "011101" => MS <= x"2" ;LS <= x"9" ;
        when "011110" => MS <= x"3" ;LS <= x"0" ;
        when "011111" => MS <= x"3" ;LS <= x"1" ;
        when "100000" => MS <= x"3" ;LS <= x"2" ;
        when "100001" => MS <= x"3" ;LS <= x"3" ;
        when "100010" => MS <= x"3" ;LS <= x"4" ;
        when "100011" => MS <= x"3" ;LS <= x"5" ;
        when "100100" => MS <= x"3" ;LS <= x"6" ;
        when "100101" => MS <= x"3" ;LS <= x"7" ;
        when "100110" => MS <= x"3" ;LS <= x"8" ;
        when "100111" => MS <= x"3" ;LS <= x"9" ;
        when "101000" => MS <= x"4" ;LS <= x"0" ;
        when "101001" => MS <= x"4" ;LS <= x"1" ;
        when "101010" => MS <= x"4" ;LS <= x"2" ;
        when "101011" => MS <= x"4" ;LS <= x"3" ;
        when "101100" => MS <= x"4" ;LS <= x"4" ;
        when "101101" => MS <= x"4" ;LS <= x"5" ;
        when "101110" => MS <= x"4" ;LS <= x"6" ;
        when "101111" => MS <= x"4" ;LS <= x"7" ;
        when "110000" => MS <= x"4" ;LS <= x"8" ;
        when "110001" => MS <= x"4" ;LS <= x"9" ;
        when "110010" => MS <= x"5" ;LS <= x"0" ;
        when "110011" => MS <= x"5" ;LS <= x"1" ;
        when "110100" => MS <= x"5" ;LS <= x"2" ;
        when "110101" => MS <= x"5" ;LS <= x"3" ;
        when "110110" => MS <= x"5" ;LS <= x"4" ;
        when "110111" => MS <= x"5" ;LS <= x"5" ;
        when "111000" => MS <= x"5" ;LS <= x"6" ;
        when "111001" => MS <= x"5" ;LS <= x"7" ;
        when "111010" => MS <= x"5" ;LS <= x"8" ;
        when "111011" => MS <= x"5" ;LS <= x"9" ;
        when OTHERS   => MS <= "XXXX" ;LS <= "XXXX" ;


    end case;

end process;

end BIN_TO_BD_ARC;
```

ד)

אכן הקוד בלוח שונה מהקוד בSSD משום שהם הפוכים.

אחד עובד בשיטת האנודה המשותפת והשני בשיטת הקטודה המשותפת.

4) דיאגרמת בלוקים:



I/0 signal	Internal signals	Direction	Description
CLOCK		in	כניסה בעלת תדירות 100 מגה הרץ
RESET		in	ריסט אנסינכרוני פעיל באחד
TEST_MODE		in	אם (1) => מונה עד 10 אם (0) => מונה עד 100
BASYS_ANODE_OUT[3:0]		out	שולט על כל ספרה בלוח
BASYS_SEGMENTS_OUT[6:0]		out	מחליט איזו ספרה להפעיל
SSD_CATODE_OUT		out	מחליט איזו ספרה להפעיל
SSD_SEGMENTS_OUT[6:0]		out	שולט על כל ספרה בלוח
	Seconds_out[5:0]		מחבר בין השעון לרכיב שמחלק לשני ספרות
	Minutes_out[5:0]		
	Hours_out[4:0]		
	Hours_out_fix[4:0]		
	Ms_seconds[3:0]		מחבר בין רכיב מחלק הספרות למקודד של הלוח
	Ls_seconds[3:0]		
	Ms_minutes[3:0]		
	Ls_minutes[3:0]		
	Ms_hours[3:0]		מחבר בין רכיב מחלק הספרות למקודד של SSD
	Ls_hours[3:0]		

מחבר בין המקודד למוקס		Seg_ms_sec[6:0]	
		Seg_ls_sec[6:0]	
		Seg_ms_min[6:0]	
		Seg_ls_min[6:0]	
		Seg_ms_ho[6:0]	
		Seg_ls_ho[6:0]	
		fequency[1:0]	

XDC .6

```
##########
# Clocks #
##########
create_clock -name CLK -period 10 -waveform {0 5} [get_ports CLK]

###############
# Input Delay #
###############
set_input_delay 0 -clock CLK  -add_delay  [get_ports RESET TEST_MODE]

################
# Output Delay #
################
set_output_delay 0 -clock CLK  -add_delay  [get_ports {BASYS_SEGMENTS_OUT* SSD_SEGMENTS_OUT* BASYS_ANODE_OUT* SSD_CATHODE_OUT }]

#############################
# Pin Locations and Voltages #
#############################
set_property -dict { PACKAGE_PIN W5   IOSTANDARD LVCMOS33 } [get_ports { CLK   }];
set_property -dict { PACKAGE_PIN U18  IOSTANDARD LVCMOS33 } [get_ports { RESET }];
set_property -dict { PACKAGE_PIN T17  IOSTANDARD LVCMOS33 } [get_ports { TEST_MODE }];
set_property -dict { PACKAGE_PIN W4   IOSTANDARD LVCMOS33 } [get_ports { BASYS_ANODE_OUT[0]  }];
set_property -dict { PACKAGE_PIN V4   IOSTANDARD LVCMOS33 } [get_ports { BASYS_ANODE_OUT[1]  }];
set_property -dict { PACKAGE_PIN U4   IOSTANDARD LVCMOS33 } [get_ports { BASYS_ANODE_OUT[2]  }];
set_property -dict { PACKAGE_PIN U2   IOSTANDARD LVCMOS33 } [get_ports { BASYS_ANODE_OUT[3]  }];
set_property -dict { PACKAGE_PIN W7   IOSTANDARD LVCMOS33 } [get_ports { BASYS_SEGMENTS_OUT[0]  }];
set_property -dict { PACKAGE_PIN W6   IOSTANDARD LVCMOS33 } [get_ports { BASYS_SEGMENTS_OUT[1]  }];
set_property -dict { PACKAGE_PIN U8   IOSTANDARD LVCMOS33 } [get_ports { BASYS_SEGMENTS_OUT[2]  }];
set_property -dict { PACKAGE_PIN V8   IOSTANDARD LVCMOS33 } [get_ports { BASYS_SEGMENTS_OUT[3]  }];
set_property -dict { PACKAGE_PIN U5   IOSTANDARD LVCMOS33 } [get_ports { BASYS_SEGMENTS_OUT[4]  }];
set_property -dict { PACKAGE_PIN V5   IOSTANDARD LVCMOS33 } [get_ports { BASYS_SEGMENTS_OUT[5]  }];
set_property -dict { PACKAGE_PIN U7   IOSTANDARD LVCMOS33 } [get_ports { BASYS_SEGMENTS_OUT[6]  }];
set_property -dict { PACKAGE_PIN A14  IOSTANDARD LVCMOS33 } [get_ports { SSD_CATHODE_OUT }];
set_property -dict { PACKAGE_PIN A16  IOSTANDARD LVCMOS33 } [get_ports { SSD_SEGMENTS_OUT[0]  }];
set_property -dict { PACKAGE_PIN B15  IOSTANDARD LVCMOS33 } [get_ports { SSD_SEGMENTS_OUT[1]  }];
set_property -dict { PACKAGE_PIN B16  IOSTANDARD LVCMOS33 } [get_ports { SSD_SEGMENTS_OUT[2]  }];
set_property -dict { PACKAGE_PIN A15  IOSTANDARD LVCMOS33 } [get_ports { SSD_SEGMENTS_OUT[3]  }];
set_property -dict { PACKAGE_PIN A17  IOSTANDARD LVCMOS33 } [get_ports { SSD_SEGMENTS_OUT[4]  }];
set_property -dict { PACKAGE_PIN C15  IOSTANDARD LVCMOS33 } [get_ports { SSD_SEGMENTS_OUT[5]  }];
set_property -dict { PACKAGE_PIN C16  IOSTANDARD LVCMOS33 } [get_ports { SSD_SEGMENTS_OUT[6]  }];
##########################
# Configuration Settings #
##########################
set_property CFGBVS VCCO [current_design]
set_property CONFIG_VOLTAGE 3.3 [current_design]

```

.7

א) Top level

```vhdl
library IEEE;
use IEEE.STD_LOGIC_1164.All;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity Top_level is
port(
CLK                 :  in std_logic;
RESET               :  in std_logic;
TestMode            :  in std_logic;
BASYS_ANODE_OUT     :  out std_logic_VECTOR(3 downto 0);
BASYS_SEGMENTS_OUT  :  out std_logic_VECTOR(6 downto 0);
SSD_CATHODE_OUT     :  out std_logic;
SSD_SEGMENTS_OUT    :  out std_logic_VECTOR(6 downto 0)
);

end Top_level;

architecture ARC_Top_level of Top_level is

component REAL_CLK
 port
 (
     CLK          : in std_logic;
     RESET        : in std_logic;
     TestMode     : in std_logic;
     Seconds_Out  : out std_logic_VECTOR(5 downto 0);
     Minutes_Out  : out std_logic_VECTOR(5 downto 0);
     Hours_Out    : out std_logic_VECTOR(4 downto 0)
 );
 end component;

component BIN_TO_BD
 port
 (
     VALUE_BIN    : in    STD_LOGIC_VECTOR(5 downto 0);
     MS           : out   STD_LOGIC_VECTOR(3 downto 0);
     LS           : out   STD_LOGIC_VECTOR(3 downto 0)
 );
 end component;

component BINARY_TO_7SEGMENT
 port
 (
     B_IN            : in    STD_LOGIC_VECTOR(3 downto 0);
     seven_segment   : out   STD_LOGIC_VECTOR(6 downto 0)
 );
 end component;

component BINARY_TO_7SEGMENT_SSD
 port
 (
     B_IN            : in    STD_LOGIC_VECTOR(3 downto 0);
    seven_segment    : out   STD_LOGIC_VECTOR(6 downto 0)
 );
  end component;

--inertial signals--
--REAL_CLK--
signal Seconds_Out         :  std_logic_VECTOR(5 downto 0);
signal Minutes_Out         :  std_logic_VECTOR(5 downto 0);
signal Hours_Out           :  std_logic_VECTOR(4 downto 0);
signal Hours_Out_with0     :  std_logic_VECTOR(5 downto 0);
--BIN_TO_BD--
signal    MS_Seconds       :    STD_LOGIC_VECTOR(3 downto 0);
signal    LS_Seconds       :    STD_LOGIC_VECTOR(3 downto 0);
signal    MS_Minutes       :    STD_LOGIC_VECTOR(3 downto 0);
signal    LS_Minutes       :    STD_LOGIC_VECTOR(3 downto 0);
signal    MS_Hours         :    STD_LOGIC_VECTOR(3 downto 0);
signal    LS_Hours         :    STD_LOGIC_VECTOR(3 downto 0);
--BINARY_TO_7SEGMENT--
signal Sec_7segment_MS     :   STD_LOGIC_VECTOR(6 downto 0);
signal Sec_7segment_LS     :   STD_LOGIC_VECTOR(6 downto 0);
signal Min_7segment_MS     :   STD_LOGIC_VECTOR(6 downto 0);
signal Min_7segment_LS     :   STD_LOGIC_VECTOR(6 downto 0);
--BINARY_TO_7SEGMENT_SSD--
signal Hours_7segment_MS   :   STD_LOGIC_VECTOR(6 downto 0);
signal Hours_7segment_LS   :   STD_LOGIC_VECTOR(6 downto 0);
--fequency--
signal fequency            :   STD_LOGIC_VECTOR(1 downto 0);

begin
Hours_Out_with0 <= '0'&Hours_Out;

L0:  REAL_CLK                PORT map(CLK, RESET, TestMode, Seconds_Out, Minutes_Out, Hours_Out);

L1:  BIN_TO_BD               PORT map(Seconds_Out,     MS_Seconds, LS_Seconds);
L2:  BIN_TO_BD               PORT map(Minutes_Out,     MS_Minutes, LS_Minutes);
L3:  BIN_TO_BD               PORT map(Hours_Out_with0,  MS_Hours , LS_Hours  );

L4:  BINARY_TO_7SEGMENT      PORT map(MS_Seconds, Sec_7segment_MS );
L5:  BINARY_TO_7SEGMENT      PORT map(LS_Seconds, Sec_7segment_LS );
L6:  BINARY_TO_7SEGMENT      PORT map(MS_Minutes, Min_7segment_MS );
L7:  BINARY_TO_7SEGMENT      PORT map(LS_Minutes, Min_7segment_LS );

L8:  BINARY_TO_7SEGMENT_SSD  PORT map(MS_Hours, Hours_7segment_MS );
L9:  BINARY_TO_7SEGMENT_SSD  PORT map(LS_Hours, Hours_7segment_LS
process
begin
    wait until rising_edge(CLK);
    if (RESET = '1') then
        fequency <= (others => '0');
    else
        fequency <= fequency + 1;

    end if;
end process;

process(fequency, Sec_7segment_MS, Sec_7segment_LS, Min_7segment_MS, Min_7segment_LS)
 begin
  case fequency is
     when "00" =>
         BASYS_ANODE_OUT      <=  "0111";
         BASYS_SEGMENTS_OUT   <=  Min_7segment_MS;
     when "01" =>
         BASYS_ANODE_OUT      <=  "1011";
         BASYS_SEGMENTS_OUT   <=  Min_7segment_LS;
     when "10" =>
         BASYS_ANODE_OUT      <=  "1101";
         BASYS_SEGMENTS_OUT   <=  Sec_7segment_MS;
     when "11" =>
         BASYS_ANODE_OUT      <=  "1110";
         BASYS_SEGMENTS_OUT   <=  Sec_7segment_LS;
     when others =>
         BASYS_ANODE_OUT      <=  "XXXX";
         BASYS_SEGMENTS_OUT   <=  "XXXXXXX";
     end case;
end process;

process(fequency(0), Hours_7segment_MS, Hours_7segment_LS)
 begin
  case fequency(0) is
     when "0" =>
         SSD_CATHODE_OUT      <=  "0";
         SSD_SEGMENTS_OUT     <=  Hours_7segment_MS;
     when "1" =>
         SSD_CATHODE_OUT      <=  "1";
         SSD_SEGMENTS_OUT     <=  Hours_7segment_LS;
     end case;
end process;
end ARC_Top_level;
```

ב) נמצא למעלה בשאלה 3 סעיפים ב,ג

ג) REAL CLK

```vhdl
library IEEE;
use IEEE.STD_LOGIC_1164.All;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity REAL_CLK is
port(
CLK : in std_logic;
RESET : in std_logic;
TestMode : in std_logic;
Seconds_Out: out std_logic_VECTOR(5 downto 0);
Minutes_Out: out std_logic_VECTOR(5 downto 0);
Hours_Out: out std_logic_VECTOR(4 downto 0)
);

end REAL_CLK;

architecture ARC_REAL_CLK of REAL_CLK is
signal Q_Seconds   :      std_logic_VECTOR(5 downto 0);
signal Q_Minutes   :      std_logic_VECTOR(5 downto 0);
signal Q_Hours     :      std_logic_VECTOR(4 downto 0);
signal Q           :      std_logic_VECTOR(26 downto 0);
signal TC_TimeBase :      std_logic;
begin
------------------tc time base and mux for test mode
process
begin
 wait until rising_edge(CLK);
 if (RESET = '1') then
 Q <= (others => '0');
    TC_TimeBase <= '0';
 elsif (TestMode = '1') then
    if (Q < 9) then
        Q <= Q + 1;
        TC_TimeBase <= '0';
    else
        Q <= (others => '0');
        TC_TimeBase <= '1';
    end if;
 elsif (TestMode = '0') then
    if (Q < 99999999) then
        Q <= Q + 1;
        TC_TimeBase <= '0';
    else
        Q <= (others => '0');
        TC_TimeBase <= '1';
    end if;
 end if;
end process;

---------- time counting up to seconds
The_Seconds: process
begin
 wait until rising_edge(CLK);
 if (RESET = '1') then
    Q_Seconds <= (others => '0');
 else
    if (TC_TimeBase = '1')  then

        if (Q_Seconds < 59) then
           Q_Seconds <= Q_Seconds + 1;
        else
           Q_Seconds <= (others => '0');
        end if;
    end if;
 end if;
end process;
Seconds_Out <= Q_Seconds;

---------- time counting up to minutes
The_Minutes: process
begin
 wait until rising_edge(CLK);
 if (RESET = '1') then
    Q_Minutes <= (others => '0');
 else
    if ((TC_TimeBase = '1') and (Q_Seconds = 59)) then
        if (Q_Minutes < 59) then
            Q_Minutes <= Q_Minutes + 1;
        else
           Q_Minutes <= (others => '0');
        end if;
    end if;
 end if;
end process;
Minutes_Out <= Q_Minutes;

---------- time counting up to hours
The_Hours: process
begin
 wait until rising_edge(CLK);
 if (RESET = '1') then
    Q_HOURS <= (others => '0');
 else
    if ((TC_TIMEBASE = '1') and (Q_MINUTES = 59) and (Q_SECONDS = 59)) then
        if (Q_HOURS < 23) then
            Q_HOURS <= Q_HOURS + 1;
        else
            Q_HOURS <= (others => '0');
        end if;
    end if;
 end if;
end process;
Hours_Out <= Q_Hours;

end ARC_REAL_CLK;
```

(8

TESTBENCH:

```
library IEEE;
use IEEE.STD_LOGIC_1164.All;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity TB_Top_level is
end TB_Top_level;

architecture TB_ARC_Top_level of TB_Top_level is
component Top_level
port(
CLK                 :  in std_logic;
RESET               :  in std_logic;
TestMode            :  in std_logic;
BASYS_ANODE_OUT     :  out std_logic_VECTOR(3 downto 0);
BASYS_SEGMENTS_OUT  :  out std_logic_VECTOR(6 downto 0);
SSD_CATHODE_OUT     :  out std_logic;
SSD_SEGMENTS_OUT    :  out std_logic_VECTOR(6 downto 0)
);
end component;

signal TB_CLK                :   std_logic = '0' ;
signal TB_RESET              :   std_logic;
signal TB_TestMode           :   std_logic;
signal TB_BASYS_ANODE_OUT    :   std_logic_VECTOR(3 downto 0);
signal TB_BASYS_SEGMENTS_OUT :   std_logic_VECTOR(6 downto 0);
signal TB_SSD_CATHODE_OUT    :   std_logic;
signal TB_SSD_SEGMENTS_OUT   :   std_logic_VECTOR(6 downto 0);

begin
DUT : Top_level
PORT map(
CLK                 =>   TB_CLK,
RESET               =>   TB_RESET,
TestMode            =>   TB_TestMode,
BASYS_ANODE_OUT     =>   TB_BASYS_ANODE_OUT,
BBASYS_SEGMENTS_OUT =>   TB_BASYS_SEGMENTS_OUT,
SSD_CATHODE_OUT     =>   TB_SSD_CATHODE_OUT,
SSD_SEGMENTS_OUT    =>   TB_SSD_SEGMENTS_OUT
);
TB_RESET <= '1',
            '0' after 13 ns;
TB_TestMode <= '1';

process
begin
  wait for 5 ns;
  TB_CLK <= not(TB_CLK);
end process;
 -- בשביל שהקלט יפלט כל שניה --
end TB_ARC_Top_level;
```